

GLYPH MANIA — SYSTEM AUDIT & ARCHITECTURAL REVIEW

Comprehensive Diagnostic Sweep, Defect Triage, and Full-Stack Code Review

DOCUMENT VERSION

1.0.0 (Release Audit)

AUDIT TIMESTAMP

September 10, 2026

TARGET WORKSPACE

AloidTech/Glyph-mania-
v1

AUDITOR SIGN-OFF

Antigravity Lead Architect

1. Executive Summary & Health Scorecard

A full-system diagnostic inspection of the **Glyph Mania** codebase was conducted across the client application, machine learning inference pipeline, database schema, and server runtimes. The immediate operational blocker — *"Every sigil in the sigil detection system is showing unknown sigil"* — was traced to a mismatch between database UUIDs, semantic IDs, hardcoded fallback strings, and cold-start weights in the Zustand store. This issue has been definitively resolved.

Beyond this defect, the system audit revealed architectural redundancies, including a ghost Express backend, business logic duplication across three independent runtimes, and oversized monolithic UI components.

SYSTEM SUBSYSTEM	HEALTH	AUDIT STATUS	FINDINGS & PRIMARY DIAGNOSIS
Sigil Detection & Recognition	A (RESTORED)	Resolved	Universal canonical resolver deployed with multi-tiered fallback (neural network → k-NN → canonical defaults).
Backend Infrastructure	C (CRITICAL)	Action Required	Orphan Express.js server (port 5000) running with mock JSON files while frontend relies on Supabase.
Solidity & Rules Engine	D (CRITICAL)	Action Required	~600 lines of complex solidity logic triplicated across Browser (TS), Deno Edge Function, and Node server.
Frontend Component Architecture	C+ (WARNING)	Needs Refactor	Monolithic pages (<code>GlyphTestingPage.tsx</code> at 1,309 LOC, <code>SigilTrainingPage.tsx</code> at 881 LOC).
Machine Learning Pipeline	B+ (GOOD)	Operational	QuickDraw 512-dim CNN is clean with robust bounding box centering and tensor memory disposal.
Database & Storage Layer	B (GOOD)	Operational	Supabase schema is clean, but <code>saved_models</code> was persisting only to local IndexedDB.
Dependency & Asset Hygiene	B- (WARNING)	Cleanup Needed	Unused <code>react-icons</code> package in <code>package.json</code> ; duplicate <code>src/public/</code> directory.

2. Root Cause Analysis: The "Unknown Sigil" Detection Bug

Symptom: Every drawn symbol in the Atelier workshop and Glyph Testing studio reported `Unknown effector sigil`, `Unknown sigil for direction top: North Anchor`, or `Unknown sigil for form slot topLeft: Top-Left Augmentor`.

Root Cause Breakdown

- Database UUID vs. Semantic ID Mismatch:** Supabase migrated sigils with UUID primary keys (e.g., `b27411bd-2c07-42c4-bd7f-b8acdaf26cca` for Fire). However, existing glyph drafts, seeded presets, and ML predictions referenced semantic identifiers (`eff-fire` , `aug-position`) or labels (`Fire`). In `glyph_solidity.ts` , `getSigil(id)` strictly tested `s.id === id` . Consequently, lookups using semantic IDs or labels always returned `undefined` .
- Cardinal Direction Anchor Fallback Defect:** In `glyph_semantic_engine.ts` , cardinal directional arrows were recognized and segmented into top, right, bottom, and left slots. When converting to compositions, fallback labels were hardcoded to `'North Anchor'` , `'East Anchor'` , `'South Anchor'` , `'West Anchor'` . Because the database sigil is named `'Position'` , lookup failed and generated `Unknown sigil: North Anchor` .
- Uninitialized Machine Learning Weights:** `recognizeCropCanvas()` aborted with `available: false` when `savedWeights` in Zustand was empty. On clean startup, `savedWeights` is `null` , causing every slot to yield `null` labels, which immediately triggered solidity failure.
- Empty Initial Store State:** When components mounted before the asynchronous Supabase query resolved, `useAdminSigilsStore.getState().sigils` was empty (`[]`), causing 100% of sigil lookups to fail.

Remediation Deployed

- Universal Canonical Sigil Resolver (`sigils.ts`):** Built `resolveCanonicalSigil()` that resolves UUIDs, semantic IDs (`eff-fire`), element names (`fire`), cardinal labels (`North Anchor` , `top` → Position Augmentor), and form types.
- Multi-Tiered Inference Pipeline (`glyph_semantic_engine.ts`):** Cardinal directional slots now automatically resolve to the Position Augmentor. Effector and form slots fallback gracefully to exemplar k-NN or foundational presets when weights are not yet trained.
- Synchronized Backend Edge Logic (`glyph_logic.ts`):** Upgraded edge function validation to support both UUIDs and semantic aliases.

3. Full Architectural Audit & Deep-Dive

3.1 Backend & Data Architecture

Finding: The "Ghost" Express Backend Server

The project contains an active Express.js server (`backend/src/server.ts`) listening on port 5000 that reads and writes to local files (`backend/data/saves.json` , `backend/data/settings.json`).

Impact Analysis: The entire production frontend uses Supabase (`@supabase/supabase-js`) for Authentication, Database, Storage, and Edge Functions. The Express server is an abandoned legacy backend that creates port collisions, confuses deployment pipelines, and causes architectural drift.

Recommendation: Formally decommission `backend/src/server.ts` . Migrate settings persistence to a Supabase `user_settings` table.

3.2 Code Duplication Across 3 Environments

A critical architectural violation is the exact duplication of the ~600-line `isSolid()` and `analyzeSolidity()` rule set across three independent files:

- `frontend/src/lib/glyph_helpers/glyph_solidity.ts` (Vite / Browser React)
- `backend/supabase/functions/_shared/glyph_logic.ts` (Deno / Supabase Edge)
- `backend/src/lib/glyph_logic.ts` (Node.js / Express)

Risk: Any change to game balance, tolerance thresholds, or sigil requirements must be manually applied in 3 places. Any drift causes glyph saves to fail server-side validation.

3.3 Frontend Monolithic Components

Component File	Lines of Code	File Size	Architectural Violation & Separation of Concerns
<code>GlyphTestingPage.tsx</code>	1,309 LOC	55.6 KB	Contains canvas rendering, radial compass SVG graphs, error popup overlays, save glyph modal, and model exploration.
<code>SigilTrainingPage.tsx</code>	881 LOC	41.2 KB	Bundles exemplar recording, Recharts telemetry charts, TensorFlow training loops, and test canvas recognition.
<code>WorkshopSceneUi.tsx</code>	729 LOC	26.5 KB	Mixes Atelier canvas, hotbar inventory, testing ground transitions, diagnostic popups, and speech bubble overlays.

3.4 Dependency & File System Hygiene

- Dead Dependency:** `react-icons` is declared in `frontend/package.json` but has **0 imports** across the codebase. The project correctly standardized on `@phosphor-icons/react` .
- Misplaced Folder:** `frontend/src/public/` contains atlas textures. Vite serves assets strictly from `frontend/public/` ; the folder in `src/` is dead weight.

4. Prioritized Architectural Roadmap

PHASE	ACTION ITEM	PRIORITY	STATUS
Phase 1	Resolve Sigil Detection & Solidity Breakdown (UUID & alias resolver)	P0 - CRITICAL	COMPLETED
Phase 1	Multi-tiered inference fallback in <code>glyph_semantic_engine.ts</code>	P0 - CRITICAL	COMPLETED
Phase 2	Decommission Express server & mock JSON files	P1 - HIGH	Scheduled Next
Phase 2	Extract shared <code>glyph_logic</code> into isomorphic package	P1 - HIGH	Scheduled Next
Phase 2	Remove unused <code>react-icons</code> & delete <code>src/public/</code>	P2 - MEDIUM	Scheduled Next
Phase 3	Decompose <code>GlyphTestingPage.tsx</code> into atomic modular components	P2 - MEDIUM	Scheduled Next
Phase 3	Sync <code>saved_models_store</code> directly with Supabase PostgreSQL	P2 - MEDIUM	Scheduled Next